

Parte 1: Esquemas de Comunicación

Redes de Computadoras - CC308

José Mérida - 201105, Javier España - 23361
Anggelie Velásquez - 221181, Mia Fuentes - 23775

10 de julio de 2026

Introducción

Para esta primera parte del laboratorio nos organizamos en pareja para replicar, de forma remota, la manera en que se transmitía información antes de las redes de computadoras modernas.

La dinámica se dividió en tres partes. En la primera, cada quien envió mensajes al otro usando la estación de telégrafo (generando los pulsos o bits según el esquema) a través de una llamada de Zoom, compartiendo audio para que el receptor escuchara directamente el sonido. En la segunda parte repetimos el ejercicio, pero esta vez empaquetando el mensaje completo en una nota de voz que se enviaba de una sola vez, en vez de transmitirlo en tiempo real. En la tercera parte diseñamos, como grupo, un protocolo de conmutación de mensajes en el que un integrante actuó como conmutador, encargado de recibir las notas de voz de cada emisor y reenviarlas al receptor correspondiente.

A continuación se presentan los resultados de la transmisión en vivo (Morse y Baudot), seguidos de los resultados de la transmisión empaquetada, las respuestas a las preguntas del laboratorio y la explicación del protocolo que acordamos para la parte del conmutador.

Transmisión en vivo

En esta primera parte cada integrante de la pareja transmitió mensajes al otro en tiempo real, primero utilizando código Morse y luego código Baudot, sin posibilidad de repetir la transmisión una vez iniciada.

Código Morse

José Mérida

- TENGO HAMBRE → Indescifrable
- MENSAJE INSANO → MENSAJE INSANO
- SECRETOSOS → SENTERENSOSOS

Javier España

- LLEGARE MANANA → ESHELAR MANANA
- REUNION APLAZADA → AEEUNIDN APLAZADA

- VIAJE SEGURO → VIAO SEGURO

Código Baudot

Repetimos el mismo ejercicio de transmisión en vivo, esta vez utilizando código Baudot en lugar de Morse.

José Mérida

- PAN CON POLLO → PNAEONPLO
- POLLO CON PAN → PLRRONPNL
- COCA COLA ZERO → BREHH AOLL

Javier España

- FELICIDADES → CESNMNRJ
- FINALIZADO → CANIRNIA (Carniceria?)
- ESPERA TONO → EAGELHEAAA

Transmisión “Empaquetada”

Para esta parte usamos código Morse nuevamente, pero en lugar de transmitir en vivo, cada quien grabó el mensaje completo en una nota de voz que el receptor podía escuchar (y reproducir) tantas veces como necesitara antes de decodificarlo.

José Mérida

- BUSCAR LENTES → BUSCAR LENTES
- ALMUERZO LISTO → ALMUERZO LISTO
- LLAMAME LUEGO → LLAMAME LUERO

España

- TRAJE LAS LLAVES → TRAJE LAS LLAVES
- ESTOY LLEGANDO → ESTOY LLEGANDO
- REVISAS EL CORREO → REVISAS EL CORPEO

Preguntas

¿Qué esquema (código) fue más fácil de transmitir y por qué? ¿Qué esquema (código) fue más difícil de transmitir y por qué?

El protocolo que más nos facilitó la comunicación fue el código morse, a pesar que ninguno de nosotros estaba familiarizado con este sistema fue el más claro por mucho. Las pausas eran bastante más largas y los sonidos eran más fáciles de lograr discernir. Por otro lado, la transmisión Baudot tenía sonidos menos diferenciables y mucho más rápidos. Entonces fue mucho más difícil de establecer una comunicación, destacando también que los espacios entre letras no dan un descanso si no que son otro símbolo adicional para decodificar. Hablando en cuanto a facilidad de enviar un mensaje, utilizando el código Baudot simplemente teníamos que referenciar una letra por lo que fue mucho más fácil para el emisor. Utilizando código morse, el emisor tuvo que hacer un esfuerzo adicional en ubicar los códigos de las letras donde además sufrimos más errores de emisión que de recepción.

De manera resumida, para el emisor de mensaje el código Baudot es mucho más ergonómico pero es mucho más difícil establecer una comunicación adecuada utilizándolo. El código morse es directamente lo opuesto, agrega carga sobre el emisor pero transmite un mensaje más claro.

¿Qué esquema tuvo menos errores (incluir datos que lo evidencien)?

El esquema que menos errores tuvo fue el código morse, utilizando el código Baudot no fuimos capaces de descifrar un solo mensaje o tener alguna frase mínimamente reconocible. Por otro lado, tuvimos un caso utilizando código morse donde el mensaje fue decodificado a la perfección y otros 4 donde teniendo cierto contexto de la comunicación se pudo haber intuido cual era el mensaje a transmitir.

¿Qué dificultades involucra el enviar un mensaje de forma “empaquetada”?

Realmente no encontramos mayor desventaja utilizando los métodos de comunicación del laboratorio, encontramos que los mensajes enviados de manera empaquetada mejoran significativamente la habilidad de decodificar un mensaje. La principal fricción que podría existir en un caso real es la ausencia de feedback en tiempo real. En la comunicación en vivo, el receptor puede indicar de inmediato que no comprendió algo mientras que un mensaje empaquetado se envía como una totalidad antes de que el receptor lo procese. Esto implica que si el mensaje tiene algún error y no se puede descifrar por parte del receptor, no existe forma de corregirlo mientras se transmite. La única solución sería enviar otro mensaje de regreso solicitando la repetición, lo cual resulta en una mayor latencia al intentar solventar errores de comunicación, en lugar de resolverlos instantáneamente como ocurriría en un intercambio en vivo.

En nuestro caso, no establecimos alguna convención para comunicar errores y no pudimos obtener los beneficios de la comunicación en vivo y por lo tanto consideramos los mensajes empaquetados como superiores.

¿Qué posibilidades incluye la introducción de un conmutador en el sistema?

Introducir un conmutador permite que no todos los participantes necesiten estar conectados directamente entre sí. En lugar de que cada cliente tenga que coordinar una comunicación directa con cada uno de los demás, todos se conectan a un único punto central que se encarga de dirigir el mensaje al destino correcto. Esto reduce la cantidad de conexiones necesarias y centraliza la lógica de enrutamiento en un solo lugar, permitiendo además que el conmutador controle el orden y el ritmo en que se atienden los mensajes cuando llegan varias solicitudes casi al mismo tiempo.

¿Qué ventajas o desventajas se tienen al momento de agregar más conmutadores al sistema?

Agregar más conmutadores al sistema aporta varias ventajas. Primero, al dejar de depender de un único punto central para atender todas las solicitudes la carga se distribuye entre varios conmutadores. Esto evita que el conmutador se vuelva un cuello de botella, haciendo que el sistema sea más apto para manejar una mayor cantidad de usuarios. Además, se incrementa la tolerancia a fallos ya que un conmutador puede manejar las solicitudes de otro en caso que este deje de funcionar.

Por otro lado, la complejidad del sistema aumenta considerablemente. Las ventajas como carga distribuida o resistencia a fallos requieren planificación anticipada para funcionar de manera correcta. En un sistema que maneja una lógica de enrutamiento mal planificada, se pueden tener altas latencias al tener mensajes recorrer varios conmutadores de manera poco eficiente. Además, si los fallos se manejan de una manera *naïve* como puede ser tener un único nodo que se encargue de suplantar a los conmutadores fallidos, en este caso si existen fallos masivos el nodo de *backup* experimenta cargas altas y los mensajes se estarían enviando con una latencia muy alta.

En resumen, al agregar más conmutadores se puede desarrollar un sistema más robusto pero se requiere mayor planificación para poder desarrollar y mantener este sistema más complejo.

Conmutación de Mensajes - Protocolo

Protocolo utilizado:

- **Disponibilidad:** Primero, el emisor manda un breve mensaje de LISTO? para confirmar que el conmutador está activo. El conmutador responde OK o NO si no está disponible para abrir una conexión.
- **Apertura de conexión:** Tras recibir OK, la conexión queda abierta. A partir de este momento el conmutador permanece escuchando indefinidamente, sin necesidad de repetir el saludo en cada mensaje.
- **Envío del contenido:** Con la conexión abierta, el emisor envía la nota de voz con el contenido del mensaje seguida de un mensaje de texto indicando EMISOR | RECEPTOR. Esto puede repetirse cuantas veces sea necesario mientras la conexión siga viva.
- **Direccionamiento:** El conmutador lee la etiqueta EMISOR | RECEPTOR y reenvía la nota de voz al receptor correspondiente. Por defecto, asumimos que todos los clientes están siempre disponibles para recibir mensajes.
- **Confirmación de reenvío:** Después de reenviar cada nota de voz, el conmutador responde al emisor con un ENVIADO para confirmar que el mensaje fue efectivamente reenviado al receptor, dejando claro que la conexión sigue abierta y lista para el siguiente envío.
- **Cierre:** Cuando el emisor termina, manda un mensaje de CERRAR para cerrar la conexión de forma explícita. El conmutador responde ADIOS, deja de escuchar a ese emisor y queda libre para una nueva solicitud de LISTO?.

Nuestro protocolo funciona de manera similar a un *socket*: se negocia la apertura de la conexión con LISTO?/OK, esta se mantiene viva de forma indefinida mientras el emisor manda notas de voz etiquetadas con EMISOR | RECEPTOR (por ejemplo “*Tono | España*”), y se cierra explícitamente con CERRAR/ADIOS. Como asumimos que todos los clientes pueden recibir por defecto, lo único que realmente se negocia es la conexión de envío.

El destino del mensaje se determinó mediante la etiqueta EMISOR | RECEPTOR que acompañaba a cada nota de voz, el conmutador solo necesitaba leer esa línea para saber a quién reenviar el contenido. Esto mantiene el enrutamiento simple y separado del contenido del mensaje. Para no sobrecargarlo aprovechamos el modelo de conexión tipo *socket*, al exigir un LISTO? inicial y esperar el OK evitamos que un emisor empiece a mandar notas de voz cuando el conmutador está ocupado o inactivo. Sin embargo, esto sigue generando problemas al tener que atender múltiples conexiones a la vez por parte del conmutador. Otra posible solución incluiría un *handshake* al inicio de cada mensaje pero consideramos que esto sería incómodo para los clientes y agregaría demasiada latencia. Además, supongamos que un cliente quiera iniciar una conexión y luego de confirmarla el conmutador queda atento pero el cliente se demora en enviar el mensaje. Las demás personas siguen en espera a que se libere el servicio pero esto nunca sucede.

En resumen, el protocolo diseñado se enfocó en la simplicidad, aprovechando que el sistema estaba limitado a únicamente 3 clientes. Bajo esa escala, mantener conexiones abiertas de forma indefinida y evitar un *handshake* por mensaje resultó práctico y suficiente, aun cuando reconocemos que en un

sistema más grande estas mismas decisiones se volverían cuellos de botella. Priorizamos entonces un protocolo sencillo y fácil de coordinar entre pocas personas por encima de uno más robusto pero complejo. Pusimos este protocolo en práctica con los 4 integrantes del grupo, uno actuando como conmutador y los otros tres como clientes emisor/receptor, y logramos completar el intercambio de mensajes siguiendo cada uno de los pasos descritos sin mayor problema, confirmando que el diseño funciona correctamente en la práctica para este número reducido de clientes.

Conclusiones

A partir de este laboratorio pudimos comprobar de forma práctica las limitaciones que enfrentaban los esquemas de comunicación anteriores a las redes modernas. La transmisión en vivo, tanto en código Morse como en Baudot, evidenció que codificar y decodificar información en tiempo real sin posibilidad de repetición es una fuente considerable de errores, incluso en mensajes cortos y con pocos participantes. Esto nos permitió entender por qué la fiabilidad de la comunicación fue uno de los primeros problemas que debieron resolverse antes de poder pensar en escalar cualquier sistema de transmisión de datos.

Al comparar ambos códigos notamos que existe un compromiso entre la facilidad de emisión y la facilidad de recepción, algo que también se traduce a los sistemas de comunicación actuales al momento de diseñar un protocolo. Asimismo, empaquetar el mensaje completo antes de enviarlo redujo notablemente los errores de decodificación, ya que el receptor podía escuchar el contenido las veces que necesitara, a costa de perder la posibilidad de corregir errores en tiempo real como ocurría en la transmisión en vivo.

Finalmente, al diseñar nuestro propio protocolo para el conmutador reforzamos la idea de que centralizar el enrutamiento simplifica la comunicación entre varios clientes, pero introduce nuevos puntos de falla y decisiones de diseño que deben ajustarse a la escala del sistema. Las decisiones que tomamos, como evitar un *handshake* por mensaje y mantener conexiones abiertas indefinidamente, fueron razonables para un sistema de tres clientes, pero identificamos claramente cómo estas mismas decisiones dejarían de ser viables conforme el número de participantes crece, notando algunos de los problemas que las redes de computadoras modernas buscan resolver mediante mecanismos más sofisticados.